



PUC
CAMPINAS
PONTIFÍCIA UNIVERSIDADE CATÓLICA

[12504] Inteligência Artificial Computacional

Trabalho 1

Prof. Alexandre Seidy Ioshisaqui
`alexandre.ioshisaqui@puc-campinas.edu.br`

2026

Roteiro da aula

1 Problema

2 Tarefas

- Tarefa 1: Busca em largura
- Tarefa 2: Busca em profundidade limitada
- Tarefa 3: Busca em profundidade limitada iterativa

3 Avaliação

Seção atual

1 Problema

2 Tarefas

3 Avaliação

Problema

- O problema de busca deste trabalho é o mesmo que já trabalhamos nas aulas práticas anteriores: busca de números primos dentro do intervalo de 1-100.
- O programa deve pedir o estado inicial como entrada (e verificar se está dentro do intervalo válido).
- As ações possíveis, como antes, são: andar duas casas à esquerda (-2) ou andar cinco casas à direita (+5).
- A função `eh_primo()` utilizada em sala de aula pode ser reutilizada aqui.
- O grupo deve decidir como apresentar informações relevantes de depuração do programa em cada um dos casos (via `printf`).

Seção atual

1 Problema

2 Tarefas

- Tarefa 1: Busca em largura
- Tarefa 2: Busca em profundidade limitada
- Tarefa 3: Busca em profundidade limitada iterativa

3 Avaliação

Subseção atual

2 Tarefas

- Tarefa 1: Busca em largura
- Tarefa 2: Busca em profundidade limitada
- Tarefa 3: Busca em profundidade limitada iterativa

Descrições e dicas para a tarefa

- Implemente o algoritmo da **busca em largura** conforme o modo como implementamos em sala de aula. Para ajudar, utilize o código construído coletivamente em sala como ponto de partida.
- Desta vez, implemente o código utilizando vetores e laços de repetição conforme necessário.
- Configure o tamanho dos vetores com a diretiva `define`, ao invés de inicializar com valores numéricos diretamente. Inicialmente, defina este valor em 5 (para comparar com a primeira versão), mas depois modifique para verificar a mudança de comportamento do algoritmo.

Descrições e dicas para a tarefa

Implemente também:

- Validação na **transição de estados**, impedindo que a busca gere nós fora do intervalo de 1 a 100. Caso a busca tiver chegado a tentar passar para fora do intervalo, imprima uma mensagem ao final avisando sobre isso.
- Verificação de **estouro de memória**, isto é, quando a geração de novos nós não pode ser realizada por completo pois a fila/pilha já está cheia. Caso a busca tiver passado por isso, imprima uma mensagem ao final avisando sobre isso.

Subseção atual

2 Tarefas

- Tarefa 1: Busca em largura
- Tarefa 2: Busca em profundidade limitada
- Tarefa 3: Busca em profundidade limitada iterativa

Descrições e dicas para a tarefa

- Implemente o algoritmo da **busca em profundidade** conforme o modo como implementamos em sala de aula.
- Lembrando, a principal diferença entre a busca em largura e a busca em profundidade é o gerenciamento da ordem das visitas em pilha ao invés de fila.
- Aqui a verificação de nós repetidos é feita apenas nos nós do caminho, não verificamos a fronteira.
- Para a implementação deste algoritmo, caso prefiram, recomendo que sigam as dicas e os templates de implementação de código disponibilizadas nos slides seguintes.

Descrições e dicas para a tarefa

- Apesar do conceito ser mais complexo, o código deste algoritmo fica mais simples com a implementação de recursividade.
- Como o conceito é mais complexo, deixarei um template de implementação do algoritmo via recursão.
- A vantagem da versão recursiva é que não precisamos **explicitamente** gerenciar a pilha, pois a recursão **implicitamente** já executa em sequência de pilha.
- Assim, precisamos verificar apenas o caminho.

Descrições e dicas para a tarefa

- Implemente as funções `caminho_visitado()` e `busca_profundidade()` conforme esboçadas nos slides seguintes.
- A função `main()` chamará `busca_profundidade()` apenas uma vez.
- A função `busca_profundidade()` chamará tanto a si mesma (recursão) quanto a função `caminho_visitado()`.

Descrições e dicas para a tarefa

```
int caminho_visitado(int estado, int profundidade);  
/* - verifica o vetor global 'caminho' até 'profundidade'  
   - retorna 1 se o 'estado' já estiver no 'caminho'.  
   - retorna 0 caso contrário  
*/
```

Descrições e dicas para a tarefa

```
int busca_profundidade(int estado, int profundidade);  
/*  - recebe um estado e adiciona ao caminho  
    - se o estado é primo (objetivo), retorna o estado  
    - se bateu a profundidade máxima (variavel global),  
      - retorna -1  
    - calcula próximos nós 'esquerdo' e 'direito'  
    - chama 'busca_profundidade' para um de cada vez  
      - pense em como utilizar o resultado  
    - pense no que fazer após essas chamadas  
*/
```

Descrições e dicas para a tarefa

- Defina todos os valores constantes com a diretiva `define`.
- Não esqueça de realizar as verificações de memória e de validação de transição de estado.

Subseção atual

2 Tarefas

- Tarefa 1: Busca em largura
- Tarefa 2: Busca em profundidade limitada
- Tarefa 3: Busca em profundidade limitada iterativa

Descrições e dicas para a tarefa

- Modifique o algoritmo da Tarefa 2 para implementar a versão iterativa.
- Lembrando, a diferença é que aqui a profundidade máxima começa em 1 e vai sendo incrementada até que se encontre uma solução.

Seção atual

1 Problema

2 Tarefas

3 Avaliação

Avaliação

- (4 pts.) Implementação e organização do código (entrega no Canvas).
 - (2 pts.) Implementação do código.
 - (2 pts.) Documentação, comentários e organização.
- (6 pts.) Apresentação e arguição.
 - (2 pts.) Apresentar código base funcionando.
 - (2 pts.) Atender a variações de parâmetros sem erros durante a apresentação (tamanhos de vetores, estados iniciais ou outras variáveis).
 - (2 pts.) Completude, profundidade e clareza da apresentação e das explicações.

Prazos

Início	11/Setembro
Prazo final (entregas e apresentações)	09/Outubro

Regras

- Grupos de até 3 pessoas no máximo (serão registrados no Canvas).
- As apresentações serão realizadas apenas durante as aulas práticas.
- Considerarei o trabalho entregue quando o grupo tiver tanto já entregue o código no Canvas quanto já tiver apresentado o trabalho em sala.
- Trabalhos entregues com atraso serão descontados 0.5 pontos por semana de atraso.

Até a próxima aula!